

System Design Document

Comprehensive End-to-End Encrypted Architecture

Prepared for: Advanced Engineering Portfolio

May 15, 2026

Contents

1	System Overview	2
1.1	Definitive Technology Stack	2
2	Section A: The Cryptographic Protocol	3
2.1	1-on-1 Messaging (ECDH)	3
2.2	Groups and Channels Protocol (Sender Keys)	3
2.3	Media Encryption (Images and Videos)	3
3	Section B: Database & Backend API	4
3.1	Relational Schema (PostgreSQL 16)	4
3.2	Detailed API Specification	5
3.2.1	RESTful API	5
3.2.2	STOMP WebSocket API	5
4	Section C: Local-First State Machine (WatermelonDB)	7
4.1	Local Database Schema (WatermelonDB)	7
4.2	State Sync Lifecycle	8
5	Section D: Cloud Infrastructure & Docker Topology	9
5.1	Docker Architecture Components	9
5.2	docker-compose.yml Structure	9

1 System Overview

The End-to-End Encrypted (E2EE) Messaging Architecture ensures absolute data sovereignty, high scalability, and robust cryptographic isolation. Utilizing a “Local-First” paradigm combined with a Zero-Knowledge backend, the system guarantees that plaintext data never leaves the client device.

1.1 Definitive Technology Stack

- **Backend:** Java 21 (Virtual Threads), Spring Boot 3.3.x, Spring WebSocket (STOMP).
- **Databases & Infrastructure:** PostgreSQL 16 (Relational), Redis (Transient caching/presence), MinIO (Encrypted Object Storage), orchestrated via Docker Compose.
- **Frontend Monorepo:** React Native, TypeScript 5.x, Solito, Zustand, WatermelonDB (Local SQLite/IndexedDB).
- **Cryptographic Core:** C++20 implementing Curve25519 and AES-256-GCM, bridged via JSI (Mobile) and Emscripten/Wasm (Web).

2 Section A: The Cryptographic Protocol

2.1 1-on-1 Messaging (ECDH)

The shared secret S is computed entirely within the hardware/C++ boundary. For a sender (d_s) and receiver (Q_r):

$$S = d_s \cdot Q_r = d_r \cdot Q_s$$

The secret S generates a 256-bit AES session key via HKDF:

$$K_{\text{session}} = \text{HKDF-SHA256}(S, \text{salt}, \text{info})$$

2.2 Groups and Channels Protocol (Sender Keys)

Pairwise encryption ($O(N)$ operations) is highly inefficient for large groups. Instead, the application utilizes a **Sender Key** protocol.

1. **Initialization:** When Alice joins a group, she generates a random 32-byte *Chain Key* and an Ed25519 *Signature Key Pair*.
2. **Distribution:** Alice encrypts this *Sender Key State* pairwise for every existing group member using standard 1-on-1 ECDH, storing it in the backend's `group_sender_keys` table.
3. **Derivation:** For each subsequent message, Alice ratchets her Chain Key forward via HMAC-SHA256 to produce a unique *Message Key*.
4. **Transmission:** Alice encrypts the message payload with the Message Key, signs it, and broadcasts it to the server. The server fans it out. Recipients use their copy of Alice's Sender Key to decrypt.

Channels: Broadcast channels operate identically, except only Admins possess the ability to distribute Sender Keys. The STOMP layer enforces authorization to prevent unauthorized distribution.

2.3 Media Encryption (Images and Videos)

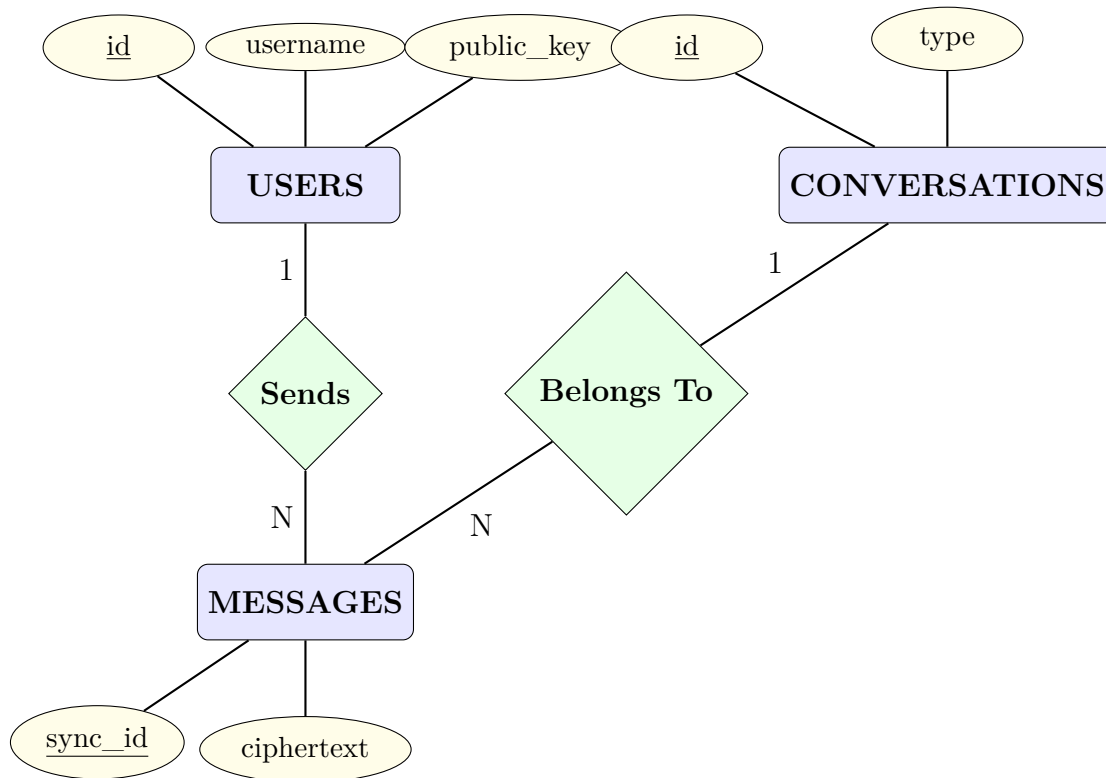
Media files (images/videos) exceed practical WebSocket payload sizes. Therefore, they follow a hybrid encryption lifecycle:

1. **Local Processing:** The UI compresses the video/image.
2. **Symmetric Encryption:** The C++ core generates a random ephemeral AES-256-GCM key (K_{media}) and encrypts the file into a 'enc' blob.
3. **Cloud Upload:** The client requests a Pre-Signed MinIO Upload URL from the Java backend and uploads the encrypted blob via HTTP PUT.
4. **Key Transmission:** The client constructs a JSON payload containing the MinIO object key and K_{media} . This payload is then encrypted using the recipient's public key (or the Group Sender Key) and sent via WebSocket.
5. **Retrieval:** The recipient decrypts the WebSocket payload, extracts K_{media} , downloads the blob from MinIO, and decrypts the media locally.

3 Section B: Database & Backend API

The backend operates on a Zero-Knowledge principle, acting solely as an orchestrated router for ciphertext.

3.1 Relational Schema (PostgreSQL 16)



PostgreSQL Detailed Schema with Referential Integrity

```
1 CREATE TABLE conversations (  
2   id UUID PRIMARY KEY,  
3   type VARCHAR(20) NOT NULL -- 'DIRECT' or 'GROUP'  
4 );  
5  
6 CREATE TABLE users (  
7   id UUID PRIMARY KEY,  
8   username VARCHAR(50) UNIQUE NOT NULL,  
9   public_key TEXT NOT NULL,  
10  last_seen TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
11 );  
12  
13 CREATE TABLE groups (  
14   conversation_id UUID PRIMARY KEY REFERENCES conversations(id),  
15   name VARCHAR(100),  
16   admin_id UUID REFERENCES users(id)  
17 );  
18  
19 CREATE TABLE group_members (  
20   group_id UUID REFERENCES groups(conversation_id),
```

```

21     user_id UUID REFERENCES users(id),
22     PRIMARY KEY (group_id, user_id)
23 );
24
25 -- Solves N x M Matrix Distribution for Sender Keys
26 CREATE TABLE group_sender_keys (
27     group_id UUID REFERENCES groups(conversation_id),
28     owner_id UUID REFERENCES users(id),
29     recipient_id UUID REFERENCES users(id),
30     encrypted_key TEXT NOT NULL,
31     PRIMARY KEY (group_id, owner_id, recipient_id)
32 );
33
34 CREATE TABLE messages (
35     sync_id BIGSERIAL PRIMARY KEY, -- Monotonic Sync Cursor
36     conversation_id UUID REFERENCES conversations(id),
37     sender_id UUID REFERENCES users(id),
38     ciphertext BYTEA NOT NULL, -- Natively encapsulates [IV +
39     Ciphertext + Tag]
40     timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
41 );

```

3.2 Detailed API Specification

The backend provides REST interfaces for key management and MinIO orchestration, and STOMP WebSockets for real-time delivery.

3.2.1 RESTful API

Endpoint	Method	Description
/api/v1/auth/register	POST	Registers user, stores Curve25519 public key.
/api/v1/users/{id}/key	GET	Retrieves public key for a target user for ECDH.
/api/v1/groups	POST	Creates a Group/Channel. Requires admin auth.
/api/v1/groups/{id}/join	POST	Publishes user's encrypted Sender Key to group.
/api/v1/media/upload-url	GET	Returns a pre-signed MinIO URL for a PUT request.
/api/v1/media/download-url	GET	Returns a pre-signed MinIO URL for a GET request.

3.2.2 STOMP WebSocket API

Destination	Type	Description
/app/chat.send	SEND	Client transmits pre-encrypted message payload.
/app/chat.sync	SEND	Client requests Delta Sync with monotonic BIGINT cursor.

/user/queue/messages	SUBSCRIBE	Private queue for inbound ciphertexts.
/topic/group/{id}	SUBSCRIBE	Topic for group messages. Server validates auth.
/topic/presence	SUBSCRIBE	Real-time online/offline updates via Redis Pub/Sub.

Critical Handshake Constraint: Web browser `WebSocket` APIs do not support injecting custom HTTP Headers (e.g., `Authorization: Bearer <jwt>`) during the HTTP Upgrade handshake. Therefore, the client **MUST** transmit the JWT within the STOMP `CONNECT` frame payload. The Spring Boot backend must intercept the `ChannelRegistration` to decode the JWT and establish the `Principal`.

4 Section C: Local-First State Machine (WatermelonDB)

The monorepo architecture ensures business logic consistency across platforms by leveraging WatermelonDB. As an **Observable** database, UI components react instantly to local database mutations.

4.1 Local Database Schema (WatermelonDB)

The client maintains a local schema that stores the **decrypted** plaintexts locally in a secure sandbox.

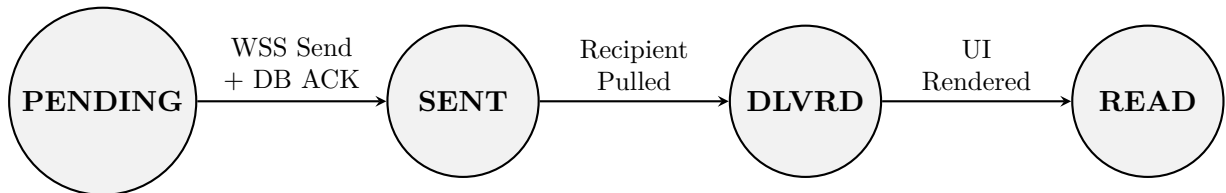
WatermelonDB Schema

```
1 export const schema = appSchema({
2   version: 2,
3   tables: [
4     tableSchema({
5       name: 'conversations',
6       columns: [
7         { name: 'name', type: 'string' },
8         { name: 'is_group', type: 'boolean' },
9         { name: 'unread_count', type: 'number' },
10      ]
11    }),
12    tableSchema({
13      name: 'messages',
14      columns: [
15        { name: 'conversation_id', type: 'string', isIndexed: true },
16        { name: 'text', type: 'string', isOptional: true },
17        { name: 'media_path', type: 'string', isOptional: true }, //
18          Local URI
19        { name: 'status', type: 'string' }, // PENDING, SENT, DELIVERED
20          , READ
21        { name: 'sync_id', type: 'number', isIndexed: true }, //
22          Monotonic Cursor
23      ]
24    }),
25    tableSchema({
26      name: 'local_sender_keys',
27      columns: [
28        { name: 'group_id', type: 'string', isIndexed: true },
29        { name: 'sender_id', type: 'string', isIndexed: true },
30        { name: 'encrypted_key_material', type: 'string' },
31        { name: 'ratchet_step', type: 'number' },
32        { name: 'last_rotated_at', type: 'number' }
33      ]
34    })
35  ]
36 });
```

The Local Sender Key Vault: To guarantee a zero-latency, offline-first experience for multi-party End-to-End Encryption, the client must avoid querying the orchestration layer for key material upon every incoming group message. When the C++ Cryptographic Core receives a new Sender Key from a peer, it is immediately persisted to the

`local_sender_keys` table. Subsequent incoming ciphertexts from that `sender_id` in that `group_id` are decrypted synchronously using the locally cached `encrypted_key_material`, incrementing the `ratchet_step` locally to maintain Perfect Forward Secrecy without requiring network calls.

4.2 State Sync Lifecycle



1. **Offline Drafting:** User writes a message. The UI writes to WatermelonDB locally with `status = PENDING`.
2. **Delta Sync:** Upon network recovery, the client calls `/app/chat.sync` providing its highest known local `sync_id`. The server streams all missing messages sequentially. The client also transmits any locally queued **PENDING** messages.
3. **Reconciliation:** Upon Server ACK, WatermelonDB updates the local record to **SENT**.

5 Section D: Cloud Infrastructure & Docker Topology

The infrastructure is deeply containerized using Docker and Docker Compose to emulate a scalable cloud environment.

5.1 Docker Architecture Components

- **Spring Boot Application (API & WSS):** Stateless orchestration layer. Can be horizontally scaled behind a load balancer (e.g., NGINX or AWS ALB).
- **PostgreSQL 16 (Persistence):** Stores metadata and ciphertext. Uses volume mounts for data persistence.
- **Redis 7 (In-Memory State):** Manages high-frequency typing indicators and WebSocket presence. Spring Boot uses Redis Pub/Sub to synchronize WebSocket sessions across multiple scaled instances.
- **MinIO (Object Storage):** An S3-compatible self-hosted bucket. Configured to issue Pre-Signed URLs, allowing the client to upload/download encrypted blobs without the media ever routing through the Java Heap memory.

5.2 docker-compose.yml Structure

Service Network Topology

```
1 version: '3.8'
2 services:
3   postgres:
4     image: postgres:16
5     ports: ["5432:5432"]
6     volumes: [pgdata:/var/lib/postgresql/data]
7
8   redis:
9     image: redis:7-alpine
10    ports: ["6379:6379"]
11
12   minio:
13     image: minio/minio
14     ports: ["9000:9000", "9001:9001"] # API & Console
15     command: server /data --console-address ":9001"
16     volumes: [miniodata:/data]
17
18   backend:
19     build: ./spring-backend
20     ports: ["8080:8080"]
21     depends_on:
22       - postgres
23       - redis
24       - minio
25     environment:
26       - SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/chatdb
27       - SPRING_REDIS_HOST=redis
28       - MINIO_ENDPOINT=http://minio:9000
29
30   volumes:
```

```
31 pgdata:
32 miniodata:
```